E-COMMERCE PLATFORM CASE STUDY

Recruiter Interview Prep & Technical Bible

A comprehensive reference handbook covering Medallion Architecture, Dimensional Star Schema Modeling, Clustered Customer Segmentations, Sales Forecasting, Basket co-occurrences, and typical interview questions.

Project Scale & Volume Specifications:

- Total Transactional Rows Processed: 33,819,106 items
- Active Customer Accounts Modeled: 206,209 shoppers
- Product Catalog Size Structured: 49,688 unique items
- Datasets Ingested: 6 tables (700MB raw CSV)

Platform: DuckDB Warehouse & Polars Streaming Ingestion

Prepared for Portfolio Showcase (Analytics & ML Careers)

1. The "Explain Like I'm 5" Glossary of Terms

Here are simple, intuitive explanations of the core technologies and terms used in this project. Use these to explain the concepts to non-technical recruiters.

- **Medallion Architecture:** Like a water filter. Raw data (Bronze) is dirty. Cleaned data (Silver) filters out formatting errors and nulls. Business data (Gold) structures it into tables for reports and models.
- **Polars:** A python data engine written in Rust. Traditional Pandas processes tables using a single thread, causing crashes on huge datasets. Polars splits work across all CPU cores, executing streams in parallel.
- **Parquet File Format:** A columnar file format. In CSV, the program must scan columns 1-9 to read column 10. In Parquet, it reads only column 10 directly. It also compresses files heavily, shrinking 700MB to 150MB.
- **DuckDB:** An analytical (OLAP) database that runs inside our Python code. Standard databases like PostgreSQL require network calls. DuckDB runs locally and is optimized for scanning millions of rows to compute averages and sums in milliseconds.
- **Star Schema:** Organizing database tables like a star. A central numeric 'Fact table' (order items) joins to slim, descriptive surrounding 'Dimension tables' (product catalog, calendar, customer segments), preventing data duplication.
- **Vectorization:** Running mathematical calculations on an entire array of numbers at once using hardware, rather than scanning rows one-by-one with a Python loop. It speeds up operations by 100x.

2. The Raw Data Anatomy & Code Walkthrough

The Raw Data Sources

- **orders.csv:** Contains 3.4M orders with user_id, order_dow (day), order_hour_of_day, and days_since_prior_order.
- **order_products (prior/train):** Contains 33.8M rows linking order_id to product_id, including add_to_cart_order and reordered flags.
- **products, aisles, departments:** Catalog maps linking product IDs to text descriptions.

The Staggered Calendar Solution

Instacart has no absolute dates. To build demand forecasts, we anchored each user's starting point to a staggered date in 2025 and added their cumulative days-since-prior-order values. This constructed a continuous calendar timeline from Jan 2025 to June 2026.

Pipeline Code Walkthrough

- **ingest_raw.py:** Copies the CSV files, enforces integer column casts, and writes them to Snappy-compressed Parquet files (Bronze).
- **medallion_pipeline.py:** Performs Silver cleaning (resolving nulls, building absolute timestamps) and streams Gold fact/dimension loads into DuckDB using Polars.
- **train_segmentation.py:** Scales and normalizes Recency, Frequency, and Monetary (RFM) columns and trains K-Means (n=4) to cluster customer buyer segments.
- **train_forecasting.py:** Aggregates daily revenue, trains Holt-Winters and RandomForest models, selects the best model on test RMSE, and forecasts 30 days out.
- **anomaly_detection.py:** Trains Isolation Forest to flag unusual sales days.
- **train_recommendations.py:** Calculates basket association rules and pre-computes collaborative recommendations using Cosine Similarity.

3. The Calculated Metrics Bible

Recruiters want to see that you understand the business context behind the numbers. Here are the core metrics we calculated and their strategic purpose:

- **Average Order Value (AOV):** Total Revenue divided by Total Orders. Tells you the average checkout size. Increasing AOV is a key way to grow gross merchandise value (GMV).
- **Repeat Purchase Rate (RPR):** Customers with more than 1 order divided by total customers. In our case, the Instacart cohort consists entirely of returning users, yielding a mathematical 100% RPR.
- **ABC Inventory Classification:** Categorizes products into A (top 80% revenue), B (next 15%), and C (bottom 5%). Out-of-stock on Class A items severely hurts revenue. Class C items represent the long-tail catalog and can be stocked minimally to save storage cost.
- **Lift (Association Rules):** Calculates if buying item A increases the likelihood of buying item B. Lift > 1 means they are highly complementary. Operations can bundle these items to increase cart size.
- **Cosine Similarity:** Measures the angle between product vectors based on user buying overlap. We use this to recommend similar products to a user.
- **Contamination (Isolation Forest):** The fraction of historical days expected to be anomalies. We configured 1%, isolating the top 6 extreme sales deviations.
- **RMSE & MAE (Forecasting):** Root Mean Squared Error and Mean Absolute Error. Measures the deviation between predicted and actual sales on the test set. Lower values indicate a better fit.

4. Recruiter Interview Simulation (Questions 1 - 6)

Q: Your dataset has 33.8 million rows. How did you process it locally without running out of memory?

Senior Response: Standard Pandas loads data as a single-threaded process, which would cause an Out-Of-Memory (OOM) crash. To solve this, I used Polars and Parquet. Polars is written in Rust and utilizes parallel execution across all CPU cores. I also used Polars' streaming execution (sink_parquet) which processes joins in chunks keeping memory usage under 200MB. I then loaded the final structured tables into DuckDB, which runs locally and aggregates millions of rows in milliseconds.

Q: Why did you use a Star Schema instead of keeping everything in one flat table?

Senior Response: A single flat table with 33.8M rows would duplicate product names, aisle descriptions, and customer segments millions of times, wasting disk space and slowing queries. By modeling the warehouse into a Star Schema, I normalized the data: storing descriptive dimensions once, and referencing them via integer Foreign Keys in a central Fact table (fact_orders). This layout maximizes DuckDB's columnar layout, making aggregations extremely fast.

Q: What was the biggest technical challenge you faced, and how did you resolve it?

Senior Response: The biggest challenge was macOS library compatibility with XGBoost. XGBoost requires the OpenMP compiler runtime (libomp.dylib), which is absent by default on macOS, causing Python to crash. To make this production-grade, I wrapped the import in a dynamic exception block. If XGBoost fails to load, the system catches the error, outputs a warning log, and falls back to training Scikit-learn's RandomForestRegressor. This guarantees zero-error deployment.

Q: Why did you use Holt-Winters and RandomForest instead of deep learning models like LSTM?

Senior Response: Deep learning models require massive computational resources and are slow to train. Classical models like Holt-Winters and tree-based ensembles (RandomForest) are industry standards for daily retail forecasting because they are highly interpretable, train in seconds, and capture seasonality. I also optimized Holt-Winters by enabling trend damping and clipping predictions to 0 to prevent negative revenue projections.

5. Recruiter Interview Simulation (Questions 7 - 11)

Q: What is the difference between your RFM and K-Means segmentation?

Senior Response: Rule-based RFM applies human-defined scoring boundaries (e.g. sorting recency into 5 equal buckets). This is great for standard marketing classifications. However, it doesn't account for complex, non-linear relationships. K-Means clustering is a data-driven, unsupervised machine learning approach. It looks at the standardized RFM space and mathematically groups customers based on spatial distance, uncovering hidden segments.

Q: Why did you scale your RFM features before training K-Means?

Senior Response: K-Means uses Euclidean distance. Features with larger ranges dominate the calculations. Monetary spend ranges from $10 to thousands, whereas Frequency ranges from 1 to 100 orders, and Recency ranges from 0 to 365 days. If we do not scale, the model will cluster customers almost entirely based on Monetary value. Standardizing the features using StandardScaler centers them, ensuring all dimensions contribute equally.

Q: How does your recommendation engine scale if the product catalog grows to 500,000 items?

Senior Response: A full product-to-product join on 500,000 items would be 250 billion pairs, causing a crash. To scale, I applied two constraints: First, in Market Basket Analysis, I filtered for the top 2,000 most popular items before running the Polars self-join. Second, in collaborative filtering, I represented user-product interactions as a Scipy Sparse Matrix (csr_matrix), which only stores non-zero values, minimizing memory footprint.

Q: Explain the business value of the ABC classification you built.

Senior Response: The ABC classification separates inventory based on revenue contribution. Our data shows that Class A items represent 9.8% of the catalog but drive 80.0% of revenue. This tells operations that a stockout on any Class A item will severely impact overall revenue, so safety stock levels must be prioritized. For Class C items (68.7% of catalog driving 5% of sales), we can keep stock minimal to reduce warehousing costs.

Q: How did you handle the lack of absolute dates in the Instacart dataset?

Senior Response: The raw dataset only provides relative days-of-week and hours-of-day. To build a realistic BI and forecasting platform, I simulated absolute calendar dates. In the Silver pipeline stage, I anchored each customer's first order to a starting date in 2025. Then, using Polars, I calculated the cumulative sum of days_since_prior_order grouped by user, and added this offset to their start date, constructing a timeline from Jan 2025 to June 2026.

6. Recruiter Interview Simulation & Production

Q: How does your Anomaly Detection model help the operations team?

Senior Response: We trained an Isolation Forest on daily order counts and revenue, flagging 6 historical anomalies. For operations, an anomaly represent a checkout system failure or a massive promotion surge. By automatically displaying these on the dashboard, business leaders can investigate and prevent operational downtime.

Q: Why did you build SQL Views instead of storing KPI tables directly?

Senior Response: Storing KPI tables requires running cron jobs to update them, which causes data staleness. By creating SQL Views, I defined the business logic once in SQL. When Streamlit queries the views, DuckDB executes the query on the fly using its vectorized engine, guaranteeing metrics are dynamically calculated and accurate.

Q: If you had to deploy this platform to production on cloud, what would the architecture look like?

Senior Response: To deploy this in production: (1) Raw CSVs or events would land in Google Cloud Storage or AWS S3. (2) I would package our Polars ETL pipelines inside Docker containers and run them on a serverless container scheduler like ECS or Google Cloud Run, scheduled daily via Apache Airflow. (3) I would write the Gold Parquet files into a cloud data warehouse like Snowflake or BigQuery. (4) Streamlit would be hosted on Cloud Run, pulling data from the warehouse.

Platform Deployment Best Practices

- **Do: Vectorized Joins:** Perform all large-scale calculations in database engines (DuckDB) or columnar libraries (Polars) rather than raw Python loops.
- **Do: Standardize Distances:** Always scale features (StandardScaler) before running distance-based ML models (K-Means).
- **Don't: Local MLflow in prod:** Avoid saving local experiment run folders (mlruns) in cloud production environments; use hosted trackers like SageMaker or MLflow tracking servers.